

Make functions flexible

```
def greet(name="Guest")
def add(*nums)
profile(age=25)
def user(**data)
```

Arguments decide how flexible a function feels.

Defaults, *args, keywords, and **kwargs solve different call styles.

Default arguments

Fallback values

```
def greet(name, msg="Welcome"):  
    print(msg, name)  
  
greet("Satya")  
greet("Satya", "Hello")
```

No value?

Python uses the default parameter value.

Override

Pass a new value when you need different behavior.

Prevents errors

Useful when a parameter can safely fall back.

Variable length args

***args**

```
def add(*nums):  
    total = 0  
    for n in nums:  
        total += n  
    return total
```

```
add(2, 4, 6)
```

One star

* collects extra positional arguments.

Tuple

Inside the function, nums behaves like a tuple.

Flexible count

Call with two, three, or many values.

Keyword arguments

Positional risk

```
def student(name, age):  
    print(name, age)
```

```
student(21, "Satya")  
# order is wrong
```

Readable call

```
student(age=21,  
        name="Satya")
```

```
# names map values  
# order is clear
```

Keyword arguments make calls readable and avoid position mistakes.

Keyword length args

****kwargs**

```
def profile(**data):  
    for key, value in data.items():  
        print(key, value)  
  
profile(name="Satya", age=21,  
        city="Pune")
```

Two stars

** collects named arguments.

Dictionary

Inside the function, data is a dict.

Dynamic keys

Handle name, age, city, or any extra field.

Choose the right tool

Argument map

<code>default value</code>	-> <code>optional input</code>
<code>*args</code>	-> <code>many values</code>
<code>keyword=value</code>	-> <code>readable mapping</code>
<code>**kwargs</code>	-> <code>many named values</code>

Default

Best when a missing value has a safe fallback.

*args

Best when the number of values can change.

**kwargs

Best when named fields can change.

Cheat Sheet

Python function arguments

Flexible signatures

```
def demo(a, b=10, *args, **kwargs):  
    print(a, b)  
    print(args)      # tuple  
    print(kwargs)    # dict
```

Default

Use fallback values for optional parameters.

*args

Collect extra positional values into a tuple.

Keywords

Name values at the call site for clarity.

**kwargs

Collect extra named values into a dictionary.

Save this for your Python journey.

Day 30/55 - Prepared by Satyanarayan Sen